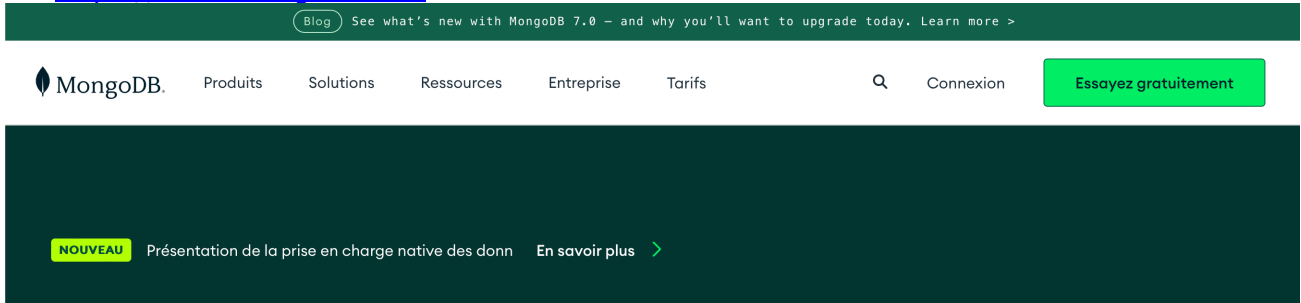


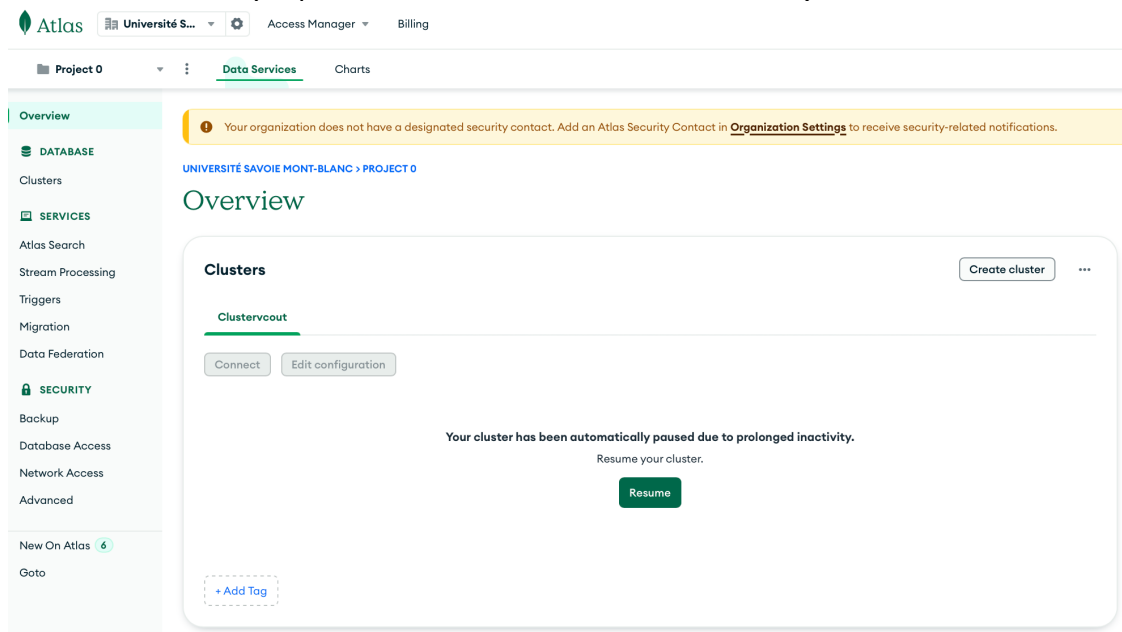
TP ramassé. Vous pouvez le faire en binôme.

1. Création/réactivation du cluster MongoDB

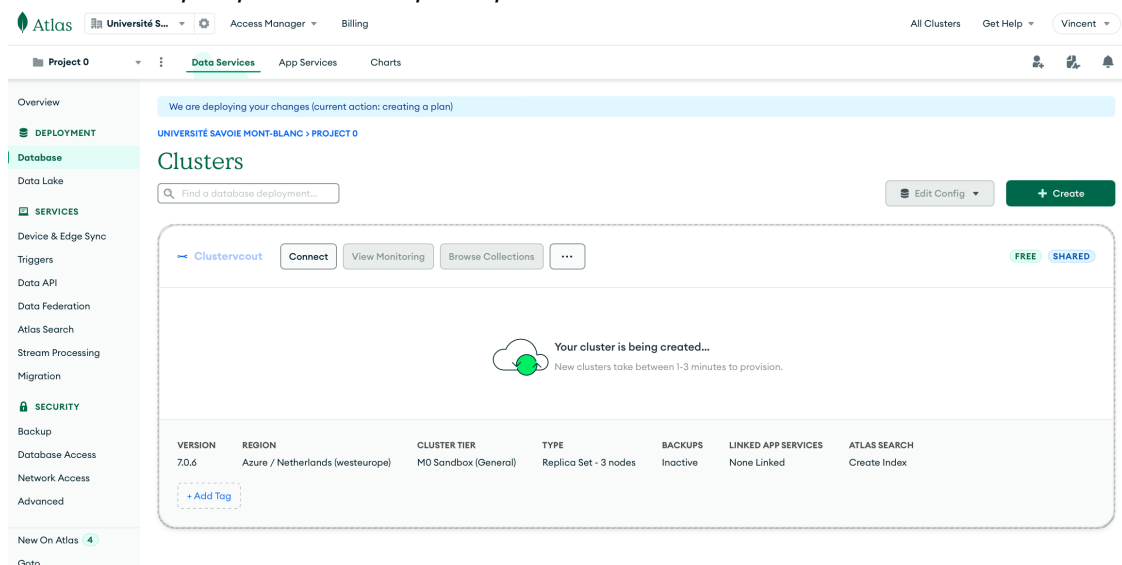
Aller sur <https://www.mongodb.com> :

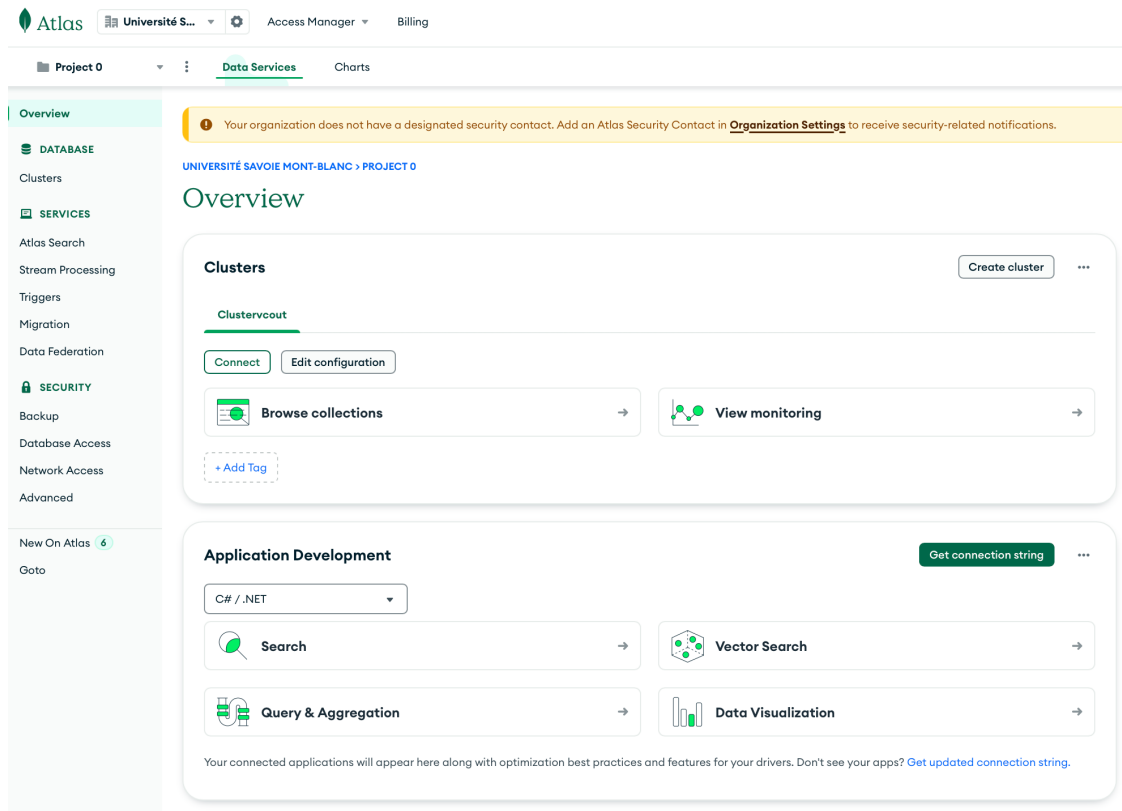


1. Si vous avez toujours accès au cluster de l'an dernier, se connecter à votre cluster puis le réactiver s'il est en mode Pause (cliquer sur le bouton au milieu de l'écran).



Il faudra ensuite quelques minutes pour que l'activation soit effective :





2. Création d'un nouveau cluster :

Cliquer sur « Essayer gratuitement » et se créer un compte sur Atlas (MongoDB en mode SaaS).

Vous recevrez un mail vous demandant de valider la création du compte (**à valider dans les 2h**).

Créer un cluster de base de données en mode **Shared** là où vous voulez. Vous pouvez renommer le cluster.

Deploy your database

Use a template below or set up advanced configuration options. You can also edit these configuration options once the cluster is created.

Template	Price	Description	Storage	RAM	vCPU
☐ M10	\$0.09/hour	For production applications with sophisticated workload requirements.	10 GB	2 GB	2 vCPUs
☐ Serverless	\$0.12/1M reads	For application development and testing, or workloads with variable traffic.	Up to 1 TB	Auto-scale	Auto-scale
☒ M0	Free	For learning and exploring MongoDB in a cloud environment.	512 MB	Shared	Shared

✓ **Free forever!** Your M0 cluster is ideal for experimenting in a limited sandbox. You can upgrade to a production cluster anytime.

Name
You cannot change the name once the cluster is created.

☒ Automate security setup ⓘ
☒ Add sample dataset ⓘ

Provider
☐ AWS
☐ Google Cloud
☒ Azure

Region

★ Recommended ⓘ ⚡ Carbon data unavailable ⓘ

Tag (optional)
Create your first tag to categorize and label your resources; more tags can be added later. [Learn more.](#)
 :

Valider la création du cluster.

Connect to ClusterVince

1

2

3

Set up connection security

Choose a connection method

Connect

You need to secure your MongoDB Atlas cluster before you can use it. Set which users and IP addresses can access your cluster now. [Read more](#)

1. Add a connection IP address

✓ Your current IP address (91.171.111.163) has been added to enable local connectivity.

2. Create a database user

This first user will have [atlasAdmin](#) permissions for this project. You'll need your database user's credentials in the next step.

We autogenerated a username and password. You can use this or create your own.

Username

couturiervinc

Password

.....

SHOW

Copy

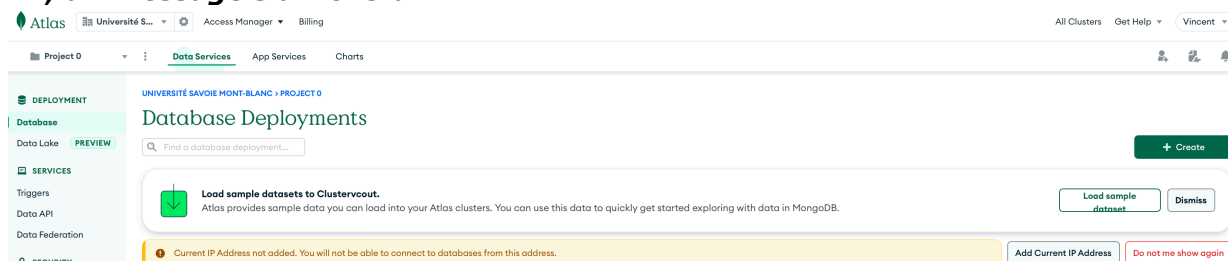
Create Database User

Cancel

Choose a connection method

Ajouter un login et un password puis cliquer sur « le bouton Create Database user ».

Remarque : si par la suite, vous utilisez une autre adresse IP pour accéder au serveur MongoDB, un message s'affichera :



Dans ce cas, il faut aller sur le menu SECURITY > Quickstart et enregistrer la nouvelle adresse IP.

SECURITY

Quickstart

Backup

Database Access

Network Access

Advanced

New On Atlas 2

Goto

My Local Environment

Use this to add network IP addresses to the IP Access List. This can be modified at any time.

Cloud Environment

Use this to configure network access between Atlas and your cloud or on-premise environment. Specifically, set up IP Access Lists, Network Peering, and Private Endpoints.

Add entries to your IP Access List

Only an IP address you add to your Access List will be able to connect to your project's clusters. You can manage existing IP entries via the [Network Access Page](#).

IP Address	Description	
Enter IP Address	Enter description	<button>Add My Current IP Address</button>
<button>Add Entry</button>		

IP Access List	Description	
91.171.111.163/32	Created as part of the Auto Setup process	EDIT REMOVE

Une fois le cluster créé ou réactivé, cliquer sur le bouton « Browse Collections ».

Atlas

Université S...

Access Manager

Billing

Project 0

Data Services

Charts

Overview

DATABASE

Clusters

Data Explorer

SERVICES

Atlas Search

Stream Processing

Triggers

Migration

Data Federation

SECURITY

Backup

Database Access

Network Access

Advanced

New On Atlas 6

Goto

Your organization does not have a designated security contact. Add an Atlas Security Contact in [Organization Settings](#) to receive security-related notifications.

UNIVERSITÉ SAVOIE MONT-BLANC > PROJECT 0

Overview

Clusters

Create cluster

...

Clustervcout

Connect

Edit configuration

Browse collections

→

View monitoring

→

+ Add Tag

Application Development

Get connection string

...

C# / .NET

Search

→

Vector Search

→

Query & Aggregation

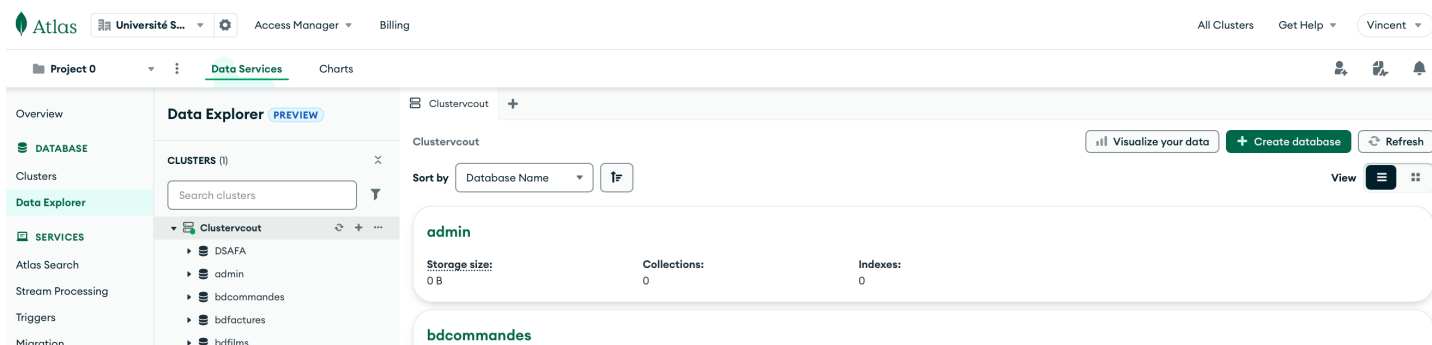
→

Data Visualization

→

Your connected applications will appear here along with optimization best practices and features for your drivers. Don't see your apps? [Get updated connection string.](#)

Activer l'installation du nouvel outil d'exploration des données.
Puis « Create Database » :



Créer la base de données `bdgames` et la collection `games` :

Create Database

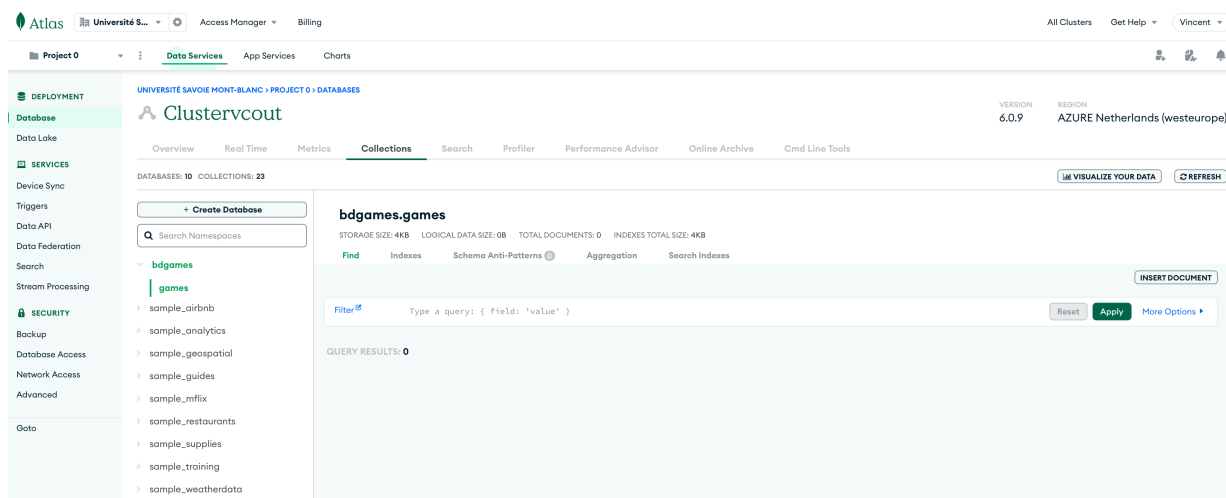
Database name ?

Collection name ?

Additional Preferences

Cancel

Create



Modèle de données de MongoDB :

- Database (= Base de données)
- Collection (= Table)
- Document (= ligne d'une table) = document BSON dont le contenu est constitué de couples clés/valeurs. Les valeurs pouvant être des entiers, réels, chaînes de caractères, des objets (dates, binaires, liens vers un document, etc.), tableaux ou un autre document, c'est à dire, pouvant à son tour contenir un autre ensemble de clés/valeurs qui peuvent aussi contenir des documents et ainsi de suite... Dans un document, des champs peuvent être ajoutés, supprimés, modifiés et renommés à tout moment.
- Field (= Colonne). Les champs d'un document sont libres et peuvent être différents d'un enregistrement à un autre au sein d'une même collection (ATTENTION cependant à ne pas faire n'importe quoi !).
- `_id` (= Clé primaire). Seul champ commun et obligatoire. Si vous ne l'écrivez pas (vous-même) dans le fichier JSON, il sera automatiquement créé.

Nous allons utiliser MongoDB Compass, le client de MongoDB, pour écrire nos requêtes.

Cliquer sur le bouton « Connect ».

UNIVERSITÉ SAVOIE MONT-BLANC > PROJECT 0

Clusters

Find a database deployment...

Clustervcouth **Connect** View Monitoring Browse Collections ... FREE

Enhance Your Experience
For production throughput and richer metrics, upgrade to a dedicated cluster now!

Upgrade

Performance Metrics:

- Read (R):** 0.02 (Last 16 minutes)
- Write (W):** 0.02/s (Last 16 minutes)
- Connections:** 5.0 (Last 16 minutes)
- In:** 48.75 B/s (Last 16 minutes)
- Out:** 407.88 B/s (Last 16 minutes)
- Data Size:** 512.00 MB

VERSION	REGION	TYPE	BACKUPS	LINKED APP SERVICES	ATLAS SQL	ATLAS SEARCH
8.0.13	Azure / Netherlands (westeurope)	Replica Set - 3 nodes	Inactive	None Linked	Connect	Create Index

[+ Add Tag](#)

Cliquer sur le bouton « Compass ».

Connect to Clustervcouth



Connect to your application

Drivers
Access your Atlas data using MongoDB's native drivers (e.g. Node.js, Go, etc.)

Access your data through tools

Compass
Explore, modify, and visualize your data with MongoDB's GUI

Shell
Quickly add & update data using MongoDB's Javascript command-line interface

MongoDB for VS Code
Work with your data in MongoDB directly from your VS Code environment

Atlas SQL
Easily connect SQL tools to Atlas for data analysis and visualization

[Go Back](#)

[Close](#)

Connect to Clustervcout

Set up connection security Choose a connection method Connect

Connecting with MongoDB Compass

I don't have MongoDB Compass installed I have MongoDB Compass installed

1. Choose your version of Compass

1.38 or later

See your Compass version in "About Compass"

2. Copy the connection string, then open MongoDB Compass

Use this connection string in your application

mongodb+srv://admin_clustervcout:<db_password>@clustervcout.eopujws.mongodb.net/

Replace <db_password> with the password for the admin_clustervcout user. Ensure any options are URL encoded. You can edit your database user password in Database Access.

RESOURCES

Connect with Compass Import and Export Data

Access your Database Users Troubleshoot Connections

Go Back Done

Copier la chaine de connexion.

Lancer Compass (menu Démarrer). Ajouter une nouvelle connexion. Coller la chaine de connexion et bien indiquer votre **mot de passe** :

New Connection

Manage your connection settings

URI Edit Connection String

mongodb+srv://admin_clustervcout:MONMOTDEPASSE@clustervcout.eopujws.mongodb.net/

Name Color

clustervcout.eopujws.mongodb.net No Color

☐ Favorite this connection

Favoriting a connection will pin it to the top of your list of connections

> Advanced Connection Options

How do I find my connection string in Atlas?

If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect. See example

How do I format my connection string?

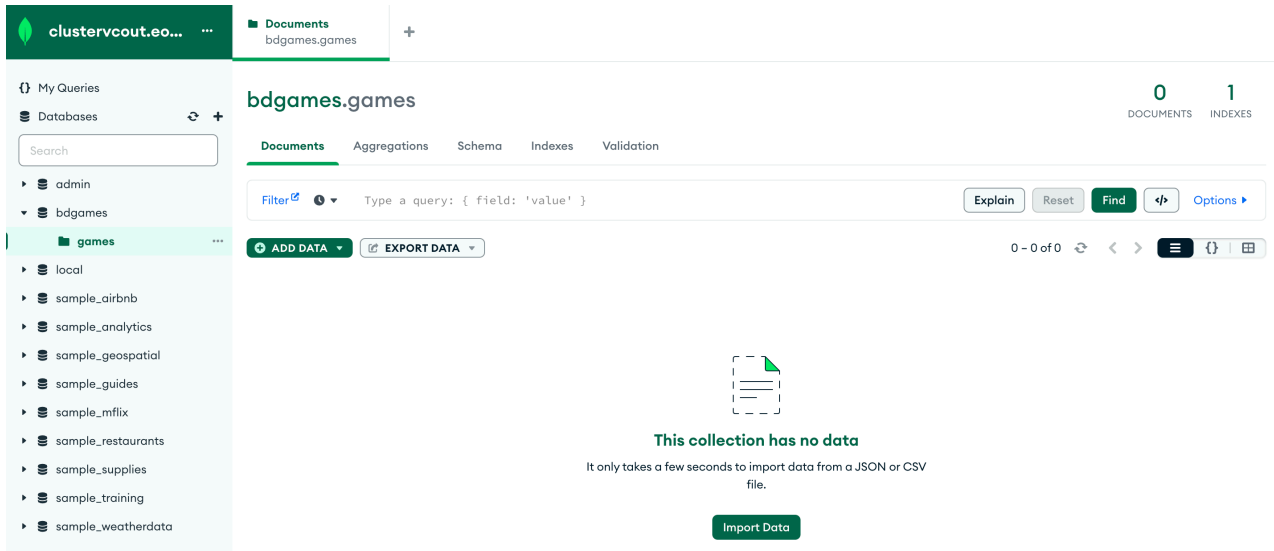
See example

Cancel Save Connect Save & Connect

Bien conserver votre chaine de connexion (avec votre mot de passe), car elle sera utile pour le TP noté.

Remarque : Compass est un outil gratuit. Pour le télécharger : <https://www.mongodb.com/try/download/compass>

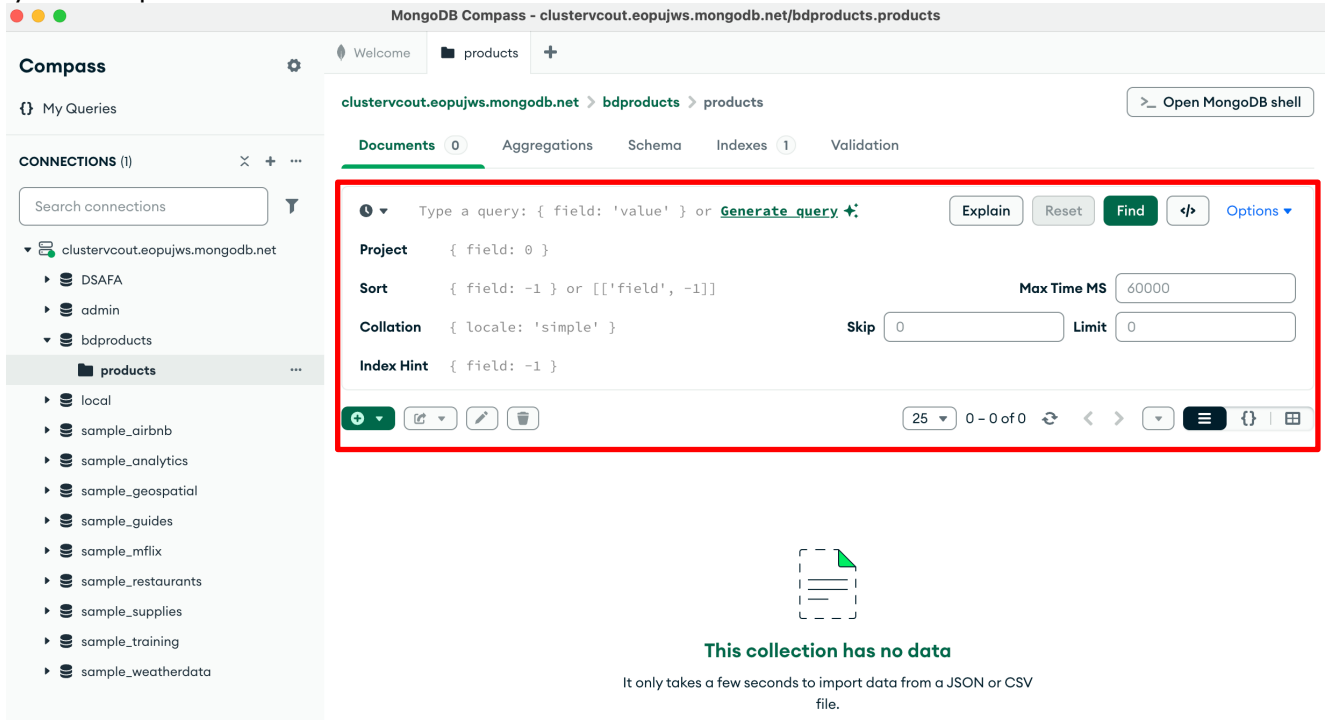
Vous retrouverez la BD créée, ainsi que la collection.



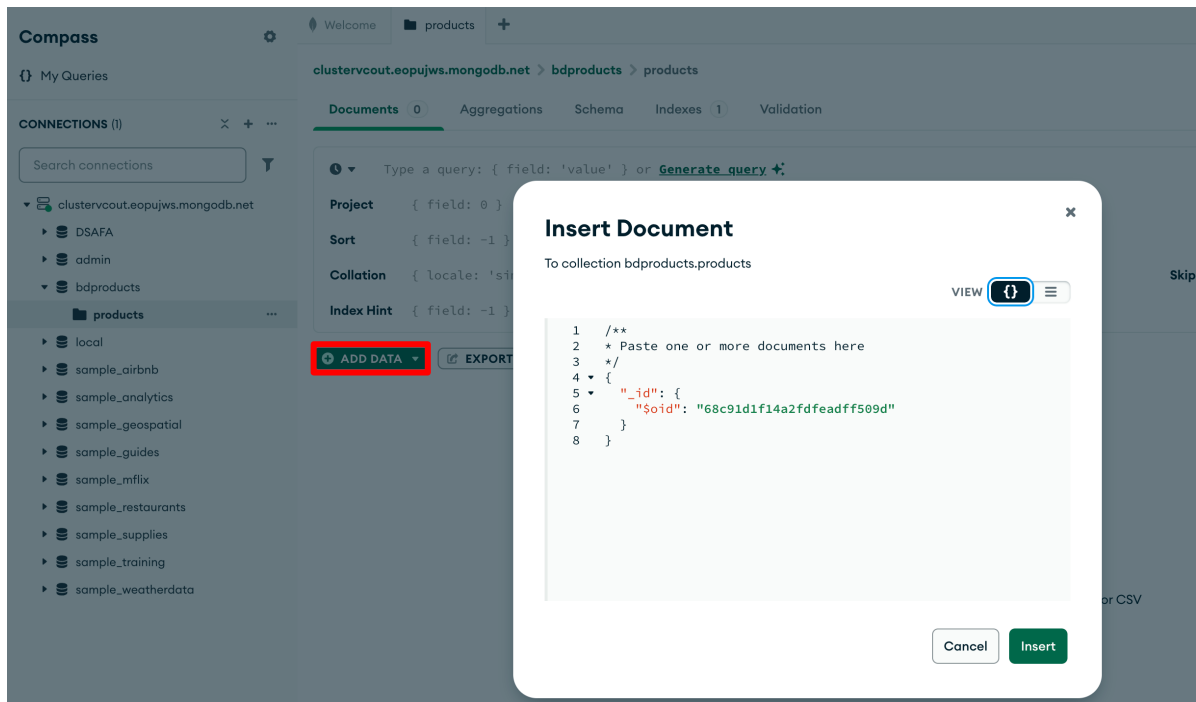
2. Manipulation des données

Dans MongoDB Compass, privilégiez la console plutôt que le query builder car il vous sera demandé l'écriture de requêtes en TP noté.

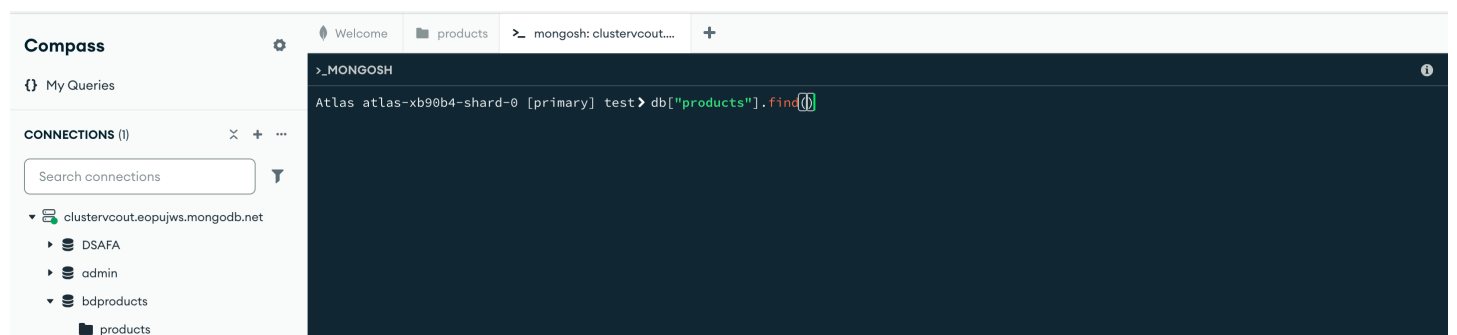
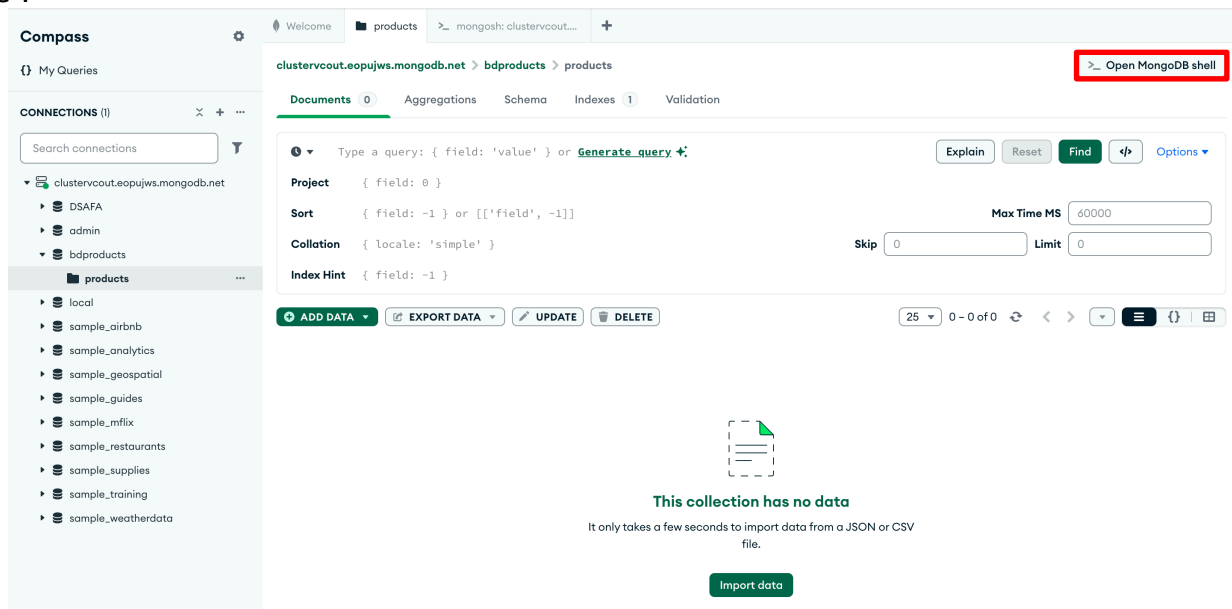
Query builder pour la recherche de documents :



Query builder pour insérer un document (NE PAS L'UTILISER) :



Console :



Remarque : dans la console, on peut exécuter les commandes suivantes (inutiles dans le client graphique) :

- `db` : pour connaître la base courante

```
>_MONGOSH
> db
< test
Atlas atlas-xb90b4-shard-0 [primary] test>
```

- `db.version()` : pour connaître la version du serveur MongoDB :

```
>_MONGOSH
> db
< test
> db.version()
< 8.0.13
Atlas atlas-xb90b4-shard-0 [primary] test>
```

- `show databases` : pour connaître les bases gérées par le serveur `mongod` sur lequel on est connecté (seules les bases non vides, i.e. contenant des collections sont affichées).
- `use NomDeLaBase` : pour choisir une base. Si la base de données n'existe pas, celle-ci sera créée au moment de l'insertion de données ou à la création manuelle d'une collection vide de documents dans celle-ci.

```
>_MONGOSH
> db
< test
> db.version()
< 8.0.13
> show databases
< DSAFA          152.00 KiB
  bdproducts      8.00 KiB
  sample_airbnb   52.34 MiB
  sample_analytics 9.46 MiB
  sample_geospatial 1.46 MiB
  sample_guides   40.00 KiB
  sample_mflix    48.56 MiB
  sample_restaurants 6.93 MiB
  sample_supplies 1.05 MiB
  sample_training 51.59 MiB
  sample_weatherdata 2.75 MiB
  admin           356.00 KiB
  local           2.03 GiB
Atlas atlas-xb90b4-shard-0 [primary] test>
```

Nous allons manipuler des documents BSON. BSON permet de gérer des types de données divers : byte, int32, int64, double, ObjectID, Timestamp, Date, tableaux, etc. (<http://bsonspec.org>).

Méthodes (principales) CRUD :

- `insertOne()` et `insertMany()`
- `find()` et `findOne()`
- `update()`
- `deleteOne()` et `deleteMany()`

Autres méthodes : <https://docs.mongodb.com/manual/reference/method/>

2.1. Insertion de données

Changer de base de données : `use bdgames`

Quand on insère des données (sous forme de document) dans une collection inexistante, MongoDB crée la collection implicitement. De même si la base de données n'existe pas.

Dans la base de données `bdgames` (`use games`), ajouter les documents suivants à la collection.

```

var game = { // Création d'un objet JSON
  "_id" : 1, // On peut aussi utiliser ObjectId('50d7423224e20473b53cae72')
  "titre" : "NBA 2K24 Edition Légende Black Mamba",
  "consoles" : ['PS5', 'PS4', 'Xbox One', 'Xbox Series X'],
  "infos" : {
    "editeur" : "Take-Two",
    "resume" : "Rassemblez votre équipe et retrouvez le passé, le présent et l'avenir
de la culture du basket dans NBA 2K24",
    "prix" : 74.99
  },
  "date sortie" : ISODate("2023-10-20T00:00:00Z")
}
// on ajoute le document game dans la collection games
db.games.insertOne(game)

```

```

>_MONGOSH
> use bdgames
< switched to db bdgames
> var game = { // Création d'un objet JSON
  "_id" : 1, // On peut aussi utiliser ObjectId('50d7423224e20473b53cae72')
  "titre" : " NBA 2K24 Edition Légende Black Mamba",
  "console" : ['PS5', 'PS4', 'Xbox One', 'Xbox Series X'],
  "infos" : {
    "editeur" : "Take-Two",
    "resume" : " Rassemblez votre équipe et retrouvez le passé, le présent et l'avenir de la culture du basket dans NBA 2K24",
    "prix" : 74.99
  },
  "date sortie" : ISODate("2023-10-20T00:00:00Z")
}
// on ajoute le document game dans la collection games
db.games.insertOne(game)
< {
  acknowledged: true,
  insertedId: 1
}
Atlas atlas-xb90b4-shard-0 [primary] bdgames>

```

`db.games.findOne()` // Retourne un seul jeu. Sensible à la casse !

```

>_MONGOSH
{
  insertedId: 1
}
> db.games.findOne()
< {
  _id: 1,
  titre: ' NBA 2K24 Edition Légende Black Mamba',
  console: [
    'PS5',
    'PS4',
    'Xbox One',
    'Xbox Series X'
  ],
  infos: {
    editeur: 'Take-Two',
    resume: " Rassemblez votre équipe et retrouvez le passé, le présent et l'avenir de la culture du basket dans NBA 2K24",
    prix: 74.99
  },
  'date sortie': 2023-10-20T00:00:00.000Z
}
Atlas atlas-xb90b4-shard-0 [primary] bdgames>

```

En mode graphique :

clustervcout.eo...

Documents

bdgames.games

My Queries

Databases

Search

admin

bdgames

games

local

sample_airbnb

sample_analytics

sample_geospatial

sample_guides

sample_mflix

sample_restaurants

sample_supplies

sample_training

sample_weatherdata

bdgames.games

0 DOCUMENTS 1 INDEXES

Documents Aggregations Schema Indexes Validation

Filter

Type a query: { field: 'value' }

Explain

Reset

Find

Options

Project

{ field: 0 }

Sort

{ field: -1 } or [['field', -1]]

MaxTimeMS 60000

Collation

{ locale: 'simple' }

Skip 0

Limit 0

ADD DATA

EXPORT DATA

1-1 of 1

Menu

JSON

Table

id: 1

titre: NBA 2K24 Edition Légende Black Mamba

console: [PS5, PS4, Xbox One, Xbox Series X]

infos: {editeur: Take-Two, resume: Rassemblez votre équipe et retrouvez le passé, le présent et l'avenir de la culture du basket dans NBA 2K24, prix: 74.99}

date sortie: {date: 2023-10-20T00:00:00.000Z}

Copy

Share

Download

Delete

clustervcout.eo...

Documents

bdgames.games

My Queries

Databases

Search

admin

bdgames

games

local

sample_airbnb

sample_analytics

sample_geospatial

sample_guides

sample_mflix

sample_restaurants

sample_supplies

sample_training

sample_weatherdata

bdgames.games

0 DOCUMENTS 1 INDEXES

Documents Aggregations Schema Indexes Validation

Filter

Type a query: { field: 'value' }

Explain

Reset

Find

Options

Project

{ field: 0 }

Sort

{ field: -1 } or [['field', -1]]

MaxTimeMS 60000

Collation

{ locale: 'simple' }

Skip 0

Limit 0

ADD DATA

EXPORT DATA

1-1 of 1

Menu

JSON

Table

id: 1

titre: NBA 2K24 Edition Légende Black Mamba

console: Array (4)

infos: Object

date sortie: 2023-10-20T00:00:00.000+00:00

Copy

Share

Download

Delete

clustervcout.eo...

Documents

bdgames.games

My Queries

Databases

Search

admin

bdgames

games

local

sample_airbnb

sample_analytics

sample_geospatial

sample_guides

sample_mflix

sample_restaurants

sample_supplies

sample_training

sample_weatherdata

bdgames.games

0 DOCUMENTS 1 INDEXES

Documents Aggregations Schema Indexes Validation

Filter

Type a query: { field: 'value' }

Explain

Reset

Find

Options

Project

{ field: 0 }

Sort

{ field: -1 } or [['field', -1]]

MaxTimeMS 60000

Collation

{ locale: 'simple' }

Skip 0

Limit 0

ADD DATA

EXPORT DATA

1-1 of 1

Menu

JSON

Table

games

_id	titre	console	infos	date sortie
1	NBA 2K24 Edition Légende ...	[4 elements	{ 3 fields	2023-10-20T00:00:00.000+00:...

Copy

Share

Download

Delete

The screenshot shows the MongoDB Compass interface. On the left, a sidebar lists databases and collections. The main area is titled 'bdgames.games' and shows a query filter, project, sort, and collation settings. Below this, a table displays the first document with fields: _id, _id ObjectId, editeur String, resume String, and prix Double.

	_id	_id ObjectId	editeur String	resume String	prix Double
1	1		"Take-Two"	" Rassemblez votre équipe e..."	74.99

Remarques :

Vous pouvez utiliser l'aide de façon suivante pour afficher les informations sur les commandes : `help`, `db.help()`, `db.<collection>.help()`, `db.<collection>.find().help()`, etc.

The screenshot shows the MongoDB Shell (MONGOSH) interface. The top part shows the help output for the `find()` method, listing various options like `addOption`, `allowDiskUse`, `allowPartialResults`, `collation`, `comment`, `count`, `hasNext`, and `hint`.

```

>_MONGOSH
> help
< Shell Help

use                               Set current database
show                               'show databases'/'show dbs': Print a list of all available databases. 'show collections'/'show tables': Print a list of all collections for current database. 'show profile': Prints system.profile information. 'show users': Print a list of all users for current database. 'show roles': Print a list of all roles for current database. 'show log <type>': log for current connection, if type is not set uses 'global' 'show logs': Print all logs.

exit                               Quit the MongoDB shell with exit/exit().exit
quit                               Quit the MongoDB shell with quit/quit()

Mongo                               Create a new connection and return the Mongo object. Usage: new Mongo(URI, options [optional])
connect                             Create a new connection and return the Database object. Usage: connect(URI, username [optional], password [optional])

it                                 result of the last line evaluated; use to further iterate
version                             Shell version
load                               Loads and runs a JavaScript file into the current shell environment

>_MONGOSH
> db.etudiants.find().help()
< Collection Cursor

addOption                           Adds OP_QUERY wire protocol flags, such as the tailable flag, to change the behavior of queries. Accepts: DBQuery.Option fields tailable, slaveOk, noTimeout, awaitData, exhaust, partial.

allowDiskUse                         Sets the 'allowDiskUse' option. If no argument is passed, the default is true.

allowPartialResults                   Sets the 'partial' option to true.

collation                             Specifies the collation for the cursor returned by the db.collection.find(). To use, append to the db.collection.find().

comment                             Adds a comment field to the query.

count                               Counts the number of documents referenced by a cursor.

hasNext                             cursor.hasNext() returns true if the cursor returned by the db.collection.find() query can iterate further to return more documents. NOTE: if the cursor is tailable with awaitData then hasNext will block until a document is returned. To check if a document is in the cursor's batch without waiting, use tryNext instead

hint                                 Call this method on a query to override MongoDB's default index selection and query optimization process. Use db.collection.getIndexes() to return the list of current indexes on a collection
  
```

Remarque : le fichier JSON précédent montre comment on peut modéliser une application simplement. On s'affranchit d'utiliser une collection pour les adresses par exemple. On parle de dématérialisation ou d'imbrication de documents (le champ adresse contient un document de 4 champs). Ici, elle a un sens car les données (adresses) sont distinctes et elle permet d'accéder rapidement aux données. Elle n'en a pas toujours quand on retrouve fréquemment les mêmes données (par exemple, même informations sur le diplôme, etc.). Se pose alors la question de la redondance et surtout de la modification (écritures multiples nécessaires).

Créer un 2^{ème} jeu.

Remarque pour insérer en masse : `insertMany` :

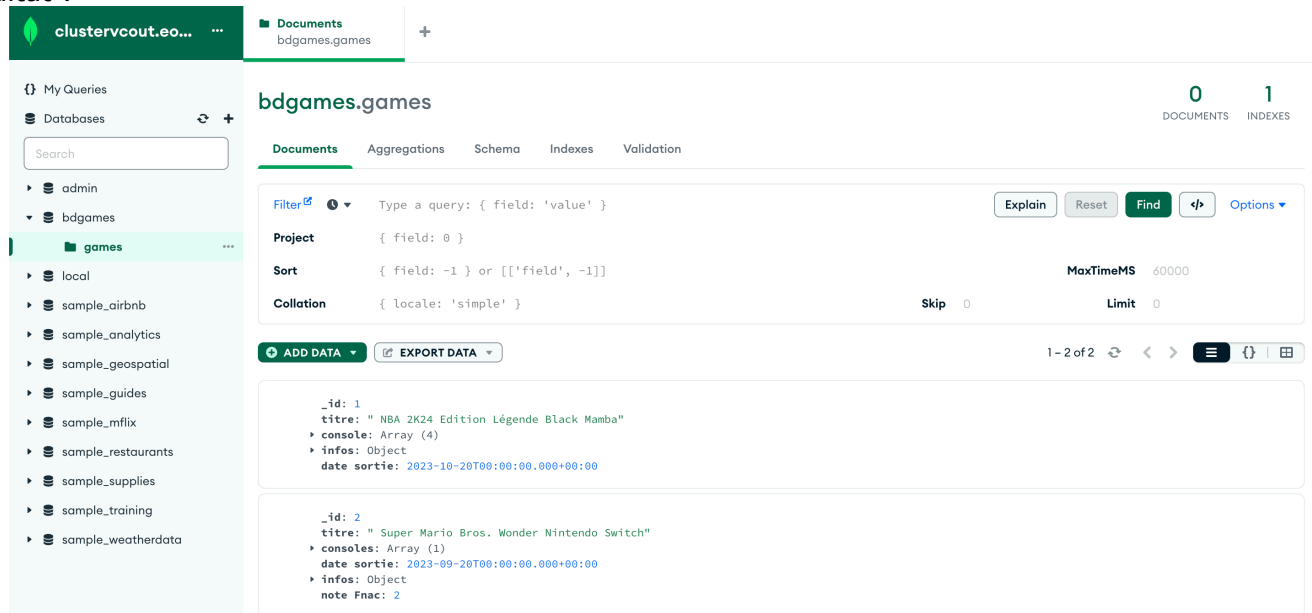
<https://docs.mongodb.com/manual/reference/method/db.collection.insertMany/>

```
var game = {
  "_id" : 2,
  "titre" : "Super Mario Bros. Wonder Nintendo Switch",
  "consoles" : ['Switch'],
  "date sortie" : ISODate("2023-09-20T00:00:00Z"),
  "infos" : {
    "editeur" : "Nintendo",
    "prix" : 49.99
  },
  "note Fnac" : 2.0
}

db.games.insertOne(game)

db.games.find()
// identique à db["games"].find() ou db.getCollection("games").find() ou
db.games.find({})
```

Résultat :



Créer un 3^{ème} jeu.

```
db.games.insertOne({
  "_id" : 3,
  "titre" : "EA Sports FC 24",
  "consoles" : ['PS4', 'PS5']
})
```

Créer un 4^{ème} jeu (sans la colonne `_id`). Vérifier que la colonne `_id` a bien été ajoutée (un `ObjectId` a été généré).

```
db.games.insertOne({
  "titre" : "Marvel's Spider-Man 2",
  "consoles" : ['PS4', 'PS5'],
  "infos" : {
    "editeur" : "Sony",
    "resume" : "Les deux Spider-Man, Peter Parker et Miles Morales, sont de retour pour une nouvelle aventure de la célèbre franchise Spider-Man de Marvel acclamée par la critique.",
    "prix" : 69.99,
    "prix adhérent FNAC" : 59.99
  },
  "notes adherents" : [3.0, 5.0, 2.0, 1.0, 5.0],
})
```

```

    "note Fnac" : 4.0
  })
}

```

ADD DATA

EXPORT DATA

1 - 4 of 4

	games					
	_id Mixed	titre String	console Array	infos Object	date sortie Date	
1	1	" NBA 2K24 Edition Légende ...	[] 4 elements	{ } 3 fields	2023-10-20T00:00:00.000+00:...	
2	2	" Super Mario Bros. Wonder ...	No field	{ } 2 fields	2023-09-20T00:00:00.000+00:...	
3	3	"EA Sports FC 24"	No field	No field	No field	
4	ObjectId('64f8e4afac99a714b...	"Marvel's Spider-Man 2"	No field	{ } 4 fields	No field	

Ajouter d'autres jeux vidéos (au minimum 3 de plus, peu importe les données). Vous pourrez utiliser <https://www.fnac.com> pour compléter votre jeu de données.

2.2. Requêtes

Rechercher un/des jeux :

```

db.games.find({'_id':2})
db.games.find({'titre':'EA Sports FC 24'})
db.games.find({'titre':"titre qui n'existe pas"})

```

Opérateurs logiques : <https://docs.mongodb.com/manual/reference/operator/query-logical/>

Opérateurs de comparaison : <https://docs.mongodb.com/manual/reference/operator/query-comparison/>

Opérateurs d'agrégat : <https://docs.mongodb.com/manual/reference/operator/aggregation-pipeline/>

Autres opérateurs : <https://docs.mongodb.com/manual/reference/operator/query/>

Ecrire des requêtes permettant de montrer l'utilisation des opérateurs précédents. Vous devrez ajouter des données.

Indications : les requêtes 1 à 18 sont des révisions ; les requêtes 19 et suivantes sont nouvelles et plus complexes...

Exemples de requêtes :

A. find()

- Afficher les titres des jeux dont le prix est ...
- Afficher les jeux dont le prix est ... sans l'ID
- Afficher les jeux dont le prix est ... ou...
- Afficher les jeux dont le prix est compris entre ... et ...
- Afficher les jeux dont la note FNAC est $\geq$ à ... ordonnés par note FNAC décroissante
- Afficher les jeux dont le ... est parmi ...
- Afficher les jeux dont le ... est ... et le ... est ...
- Afficher les jeux qui ne sont pas édités par ...
- Afficher les jeux non compatibles avec la console ...
- Afficher les jeux qui ne fonctionnent ni sur la console ... ni sur la console ...
- Afficher les jeux fonctionnant sur les consoles PS quelle que soit la version.
- Afficher les jeux qui sont édités par ... mais pas compatibles sur la console ...
- Afficher les jeux qui ont une note adhérent $<$ à ...
- Afficher les jeux dont le champ Prix Adherent FNAC n'est pas présent
- Afficher les jeux dont le champ résumé est présent
- Afficher les jeux dont le titre commence par ...

B. aggregate(), count(), distinct(), \$skip, \$unwind etc.

- Afficher les jeux à partir du numéro 3
- Trier les jeux par ... décroissant
- Afficher la note moyenne donnée par les adhérents
- Afficher la note moyenne donnée par les adhérents par jeu
- Afficher le note min (ou max) donnée par les adhérents par éditeur
- Afficher la note min (ou max) donnée par les adhérents par ... et par ... (par ex. par éditeur et par console).

23. Afficher le nombre de jeux par ...
24. Afficher le nombre de jeux dont le champ ... est présent
25. Afficher pour chaque console le nombre de jeux compatibles sur cette console. Le classement se fera par ordre décroissant (nombre).
26. Afficher le nombre de consoles par jeu vidéo.

2.3. Modification, suppression

Commande `updateOne` (<https://docs.mongodb.com/manual/tutorial/update-documents/>)

```
db.games.updateOne({'_id':2}, {$set : {"note Fnac" : 3.0}})
```

OU MIEUX, traitement en masse avec `updateMany` :

<https://docs.mongodb.com/manual/reference/method/db.collection.updateMany/>

Commande `deleteOne`

(<https://www.mongodb.com/docs/manual/reference/method/db.collection.deleteOne/>) :

```
db.games.deleteOne({'_id':2})
```

OU MIEUX, traitement en masse avec `deleteMany` :

<https://www.mongodb.com/docs/manual/reference/method/db.collection.deleteMany/>

Tester ces commandes sur plusieurs requêtes :

2.4. Modélisation des collections, jointures

Dupliquer la collection `games` en `games2` :

- Créer la collection `games2` :
- Copier les données de `games` dans `games2`. Vous pourrez utiliser `$out` : <https://www.delftstack.com/fr/howto/mongodb/mongodb-copy-collection/>

Créer la collection `pegi` (dans la même base de données).

Ajouter les informations suivantes dans la collection `pegi` :

<code>_Id</code>	<code>Code_pegi</code>	<code>Desc_pegi</code>	<code>Categorie_pegi</code>
Auto-généré	PEGI3	Adapté à toutes les classes d'âge	Enfant
	PEGI7	Scènes ou sons potentiellement effrayants	Enfant
	PEGI12	Scènes de violence imaginées	Adolescent
	PEGI16	Scènes de violence ou contacts sexuels semblables à ceux de la réalité	Adolescent
	PEGI18	Violence crue donnant un sentiment de dégoût	Adulte

1^{ère} modélisation : stockage de l'objet ou d'une partie de l'objet

Collection `games` qui contient le document (ou une partie de document) `pegi` :

```
{
  "_id" : 3,
  "titre" : "EA Sports FC 24",
  "consoles" : ['PS4','PS5'],
  "pegi" :{
    "_id" : ObjectId("60832a82ec1f4c2be651f239"),
    "desc_pegi" : "Adapté à toutes les classes d'âge"
  }
}
```

Avantages	Inconvénients
Pas besoin de jointure => rapide en lecture	Si la description PEGI change, il faut modifier tous les jeux ayant ce PEGI (par exemple « Scènes de violence imagées » -> « Scènes de violence <u>ou</u> de <u>sexe</u> imagées») => peu rapide en MaJ
	Volume de données plus important

Ajouter un champ `pegi`, contenant l'ID et la description, aux jeux vidéos. Vous pourrez utiliser la commande `db.collection.updateOne` (ou `updateMany`) pour mettre à jour un seul / plusieurs enregistrements) ou `db.collection.findAndModify` ou `db.collection.findOneAndUpdate`.

Indications :

- Vous n'êtes pas sensé connaître l'ID auto-généré du PEGI. Une première requête doit récupérer le PEGI pertinent (`_id` et `description`).

```
var pegirate=db.pegi.find(...).toArray();
```

...

Pour voir le contenu d'une variable, il suffit de faire :

```
var pegirate=db.pegi.find(...).toArray();
```

```
pegirate
```

- Une 2^{nde} requête met à jour les jeux en fonction du PEGI récupéré. Vous créez un code « dynamique » Javascript de la façon suivante :

```
var jeux = db.games.find(...).toArray();
```

```
for (var index = 0; index < jeux.length; index++) {
  switch (index%5) {
    case 0: db.games.updateOne(...)
      break;
    ...
  }
}
```

Peu importe si le PEGI ne correspond pas au jeu vidéo.

Ecrire une requête affichant les jeux (titre et consoles) et leur description PEGI.

2^{nde} modélisation : stockage de l'ID uniquement

Collection `games2` qui contient seulement l'ID des PEGI :

```
{
  "_id" : 3,
  "titre" : "EA Sports FC 24",
  "consoles" : ['PS4','PS5'],
  "pegi" :ObjectId("60832a82ec1f4c2be651f239")
}
```

Avantages	Inconvénients
Rapide en MaJ (si, par exemple, changement des infos PEGI comme la description ou la catégorie)	Peu rapide en lecture car nécessite une jointure (\$lookup)
Volume de données plus réduit	

Ajouter un champ PEGI, contenant uniquement l'ID, aux jeux vidéos. Mêmes indications que pour la section précédente.

ATTENTION, on doit obtenir ceci :

```
{
  ...
  "pegi" : ObjectId("60832a82ec1f4c2be651f239")
}
```

Et non :

```
{
  ...
  "pegi" : {
    "_id" : ObjectId("60832a82ec1f4c2be651f239")
  }
}
```

Ecrire une requête permettant de réaliser une « jointure » entre les 2 collections de 2 manières :

- En partant de `games2`
- En partant de `pegi`

Vous n'afficherez que les titre, consoles, prix et description PEGI pour chaque jeu. Vous pourrez utiliser les commandes `$lookup`, `$replaceRoot` et `$project` : <https://docs.mongodb.com/manual/reference/operator/aggregation/lookup/>

Dans la clause `$project`, vous mettrez tous les champs que vous souhaitez cacher à 0 (ou `false`).

Exemple de résultat que vous devez obtenir :

```
{
  "titre" : "EA Sports FC 24",
  "consoles" : ['PS4','PS5'],
  "description PEGI" : "Adapté à toutes les classes d'âge"
}
...
{
  "titre" : "Marvel's Spider-Man 2",
  "consoles" : ['PS4', 'PS5'],
  "prix" : 69.99,
  "description PEGI" : "Scènes de violence imaginées"
}
```

2.5. Transactions

Ecrire le code permettant de créer :

- Une transaction qui permet l'insertion :
 - D'un nouveau PEGI (ex. PEGI 70, pour les seniors)
 - ET de 2 jeux associés à ce nouveau code PEGI dans la collection `games2`
- Une transaction qui permet la modification :
 - Du PEGI « Scènes de violence imaginées » en « Scènes de violence ou de sexe imaginées »
 - ET de tous les descriptions PEGI liées dans la collection `games`

Transactions MongoDB :

- <https://dzone.com/articles/mongodb-transactions-your-very-first-transaction-w>
- <https://docs.mongodb.com/manual/core/transactions/>

- <https://www.digitalocean.com/community/tutorials/how-to-use-transactions-in-mongodb>

Le code à écrire commence à l'ouverture d'une session (`var session=db.getMongo().startSession()`).
Ajouter les copies d'écran.

3. JSON schema

Un schéma permet d'ajouter des contraintes sur les champs (contraintes de types de données, de valeurs possibles, etc.). Ce schéma peut être appliqué à la création de la collection ou plus tard. Dans ce dernier cas, il est possible que des données non compatibles avec ce schéma aient déjà été créées.

Ouvrir le fichier `movies_references_exporte.json` et regarder son contenu. Ce fichier Json contient des films.

Créer la collection `movies` dans une nouvelle base de données et y ajouter les données du fichier `movies_references_exporte.json` (cliquer sur le bouton « Add data »).

Ajouter, par exemple, les données non valides suivantes (cf. contraintes de validation ci-dessous) :

```
[
  {
    "_id": "movie:100",
    "year": 2010,
    "genre": "Science-fiction",
    "summary": "Dom Cobb est un voleur expérimenté - le meilleur qui soit dans l'art périlleux de l'extraction : sa spécialité consiste à s'approprier les secrets les plus précieux d'un individu, enfouis au plus profond de son subconscient, pendant qu'il rêve et que son esprit est particulièrement vulnérable.",
    "country": "USA",
    "director": {
      "_id": "person:150"
    },
    "actors": [
      {
        "_id": "person:230",
        "role": "Dom Cobb"
      }
    ]
  },
  {
    "_id": "movie:101",
    "title": "Acide",
    "year": 2030
  },
  {
    "_id": "movie:102",
    "title": "Inception v1",
    "year": 2010,
    "genre": "Science-fiction",
    "summary": "Dom Cobb est un voleur expérimenté - le meilleur qui soit dans l'art périlleux de l'extraction : sa spécialité consiste à s'approprier les secrets les plus précieux d'un individu, enfouis au plus profond de son subconscient, pendant qu'il rêve et que son esprit est particulièrement vulnérable.",
    "country": "UK",
    "director": {
      "_id": "person:150"
    },
    "actors": [
      {
        "_id": "person:230",
        "role": "Dom Cobb"
      }
    ]
  }
]
```

```

},
{
  "_id": "movie:103",
  "title": "Inception v2",
  "year": 2010,
  "genre": "Science-fiction",
  "summary": "Dom Cobb est un voleur expérimenté - le meilleur qui soit dans l'art périlleux de
l'extraction : sa spécialité consiste à s'approprier les secrets les plus précieux d'un
individu, enfouis au plus profond de son subconscient, pendant qu'il rêve et que son esprit est
particulièrement vulnérable.",
  "country": "USA",
  "director": {
    "_id": "person:150"
  },
  "actors": [
    {
      "_id": "person:230",
      "role": "Dom Cobb"
    },
    {
      "_id": "person:231",
      "role": "Role2"
    },
    {
      "_id": "person:232",
      "role": "Role3"
    },
    {
      "_id": "person:233",
      "role": "Role4"
    },
    {
      "_id": "person:234",
      "role": "Role5"
    },
    {
      "_id": "person:235",
      "role": "Role6"
    },
    {
      "_id": "person:236",
      "role": "Role7"
    }
  ]
}},
{
  "_id": "movie:104",
  "title": "Inception v3",
  "year": 2010,
  "genre": "Science-fiction",
  "summary": "Dom Cobb est un voleur expérimenté - le meilleur qui soit dans l'art périlleux de
l'extraction : sa spécialité consiste à s'approprier les secrets les plus précieux d'un
individu, enfouis au plus profond de son subconscient, pendant qu'il rêve et que son esprit est
particulièrement vulnérable.",
  "country": "USA",
  "director": {
    "_id": "person:150"
  },
  "actors": [
    {
      "_id": "person:230",
      "role": "Dom Cobb"
    },
    {
      "_id": "person:231",
      "role": "Role2"
    },
    {
      "_id": "person:232",
      "role": "Role3"
    },
    {
      "_id": "person:233",

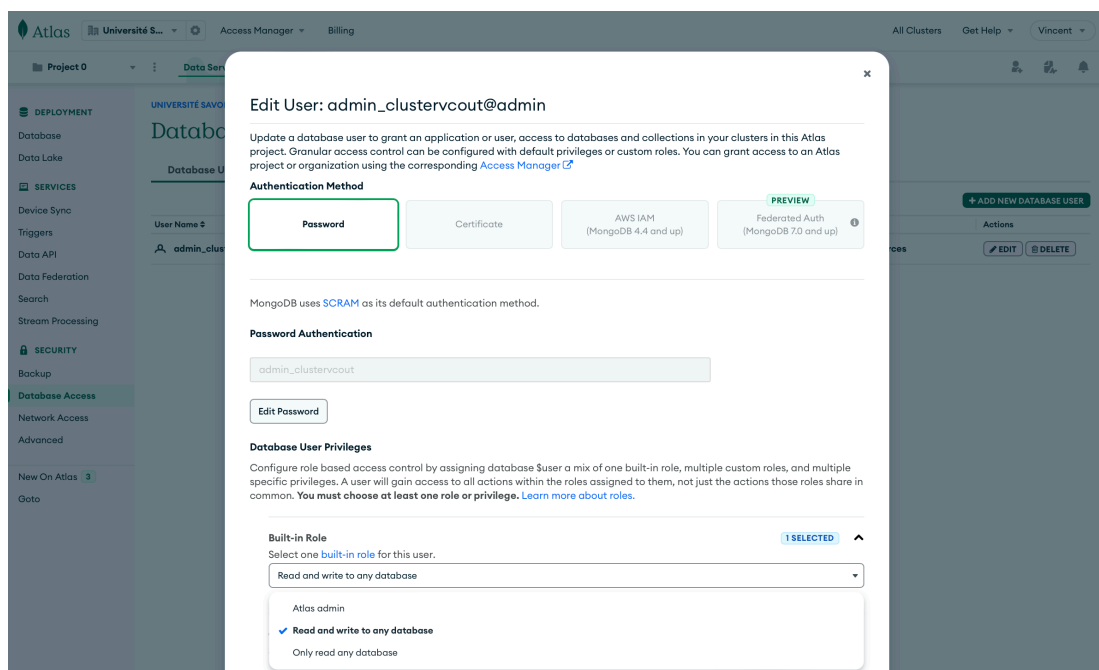
```

```

    "role": "Role4"
  },
  {
    "_id": "person:234",
    "role": "Role5"
  },
  {
    "_id": "person:235",
    "role": "Role6"
  },
  {
    "_id": "person:236",
    "role": "Role7"
  },
  {
    "_id": "person:237",
    "role": "Role8"
  }
]

```

Appliquer le schéma à la collection `movies`. Vous devrez modifier les droits de l'utilisateur (« Atlas admin ») :



En vous basant sur <https://docs.mongodb.com/manual/reference/operator/query/jsonSchema/>, réaliser le schéma du fichier Json.

Contraintes de validation à prendre en compte :

- Champs requis : `_id`, `genre`, `title`, `year`, `country`, `director`
- `actors` : tableau d'object. Nombre maximum d'acteurs dans le tableau : 6
- `year` : entre 1950 et l'année courante
- `country` : valeurs possibles : USA, FR, IT, DE, Autre

En vous basant sur :

<https://www.mongodb.com/docs/manual/core/schema-validation/use-json-schema-query-conditions/#std-label-use-json-schema-query-conditions>

écrire les commandes permettant :

1. d'afficher les documents valides
2. d'afficher les documents non valides

3. de mettre à jour des documents (pas tous) non valides :
 - mettre à jour à la valeur 2025 (année courante) les années > à 2025 (année courante)
 - modifier les pays qui ne sont pas parmi (USA, FR, UK, DE, Autre) à Autre
 - supprimer les acteurs à partir du rang n°7 (attention, votre requête devra fonctionner si 7, 8, 9 ou plus d'acteurs ont été insérés). Vous pourrez ajouter des documents pour tester votre code.
4. de supprimer des documents non valides. Vous pourrez ajouter des documents pour tester votre code.

Ajouter les copies d'écran des résultats.

Créer des documents Json ne respectant pas le schéma et qui généreront des erreurs. Ajouter les copies d'écran des erreurs obtenues.

Proposer un schéma pour la collection `games`.

Corriger les éventuelles erreurs (modification ou suppression de documents) liées aux données déjà insérées.

4. Indexation et plan d'exécution

Pour pouvoir tester l'indexation, il faut disposer de grandes quantités de données.

VERSION	REGION	TYPE	BACKUPS	LINKED APP SERVICES	ATLAS SQL	ATLAS SEARCH
8.0.15	Azure / Netherlands (westeurope)	Replica Set - 3 nodes	Inactive	None Linked	Connect	Create Index

Choisir l'un des datasets et ensuite l'une des plus grosses collections de celui-ci.

Indexation :

- <https://docs.mongodb.com/manual/reference/method/db.collection.createIndex/>
- <https://docs.mongodb.com/manual/reference/method/db.collection.getIndexes/>

- <https://docs.mongodb.com/manual/reference/method/db.collection.totalIndexSize/>
- <https://docs.mongodb.com/manual/reference/method/db.collection.reIndex/>

Explain : <https://docs.mongodb.com/manual/reference/method/db.collection.explain/>

Validation (à utiliser avant et après la création d'index pour pouvoir comparer) :

<https://docs.mongodb.com/manual/reference/method/db.collection.validate/>

Statistiques : <https://docs.mongodb.com/manual/reference/method/db.collection.stats/>

Dataset exemple choisi :

A vous de tester et d'expliquer :

Comme les SGBD relationnels, les index MongoDB sont basés sur des B-Tree. Principaux types d'index (**à expliquer ; vous pourrez ajouter du code testant ces index**) :

- Les index primaires :
-

- Les index simples :
-

- Les index composés :
-

- Les index MultiKey :
-

- Les index texte :
-

- Index géo-spatiaux (2d, 2dsphere) :
-

- Les index hachés :
-

Ces index possèdent des options :

- TTL : Time To Live.
-

- Unique :
-

- Partiel :
-

- Epars (Sparse) :
-

- Insensibilité à la casse :
-

Les index doivent pouvoir tenir en RAM dans un cluster MongoDB, afin d'avoir des performances optimales : <https://docs.mongodb.com/manual/tutorial/ensure-indexes-fit-ram/>